

Технологии программирования

Лекция 12
DevOps

**ПЕРЕЛОГИНЬТЕСЬ С ФАМИЛИЕЙ И ИМЕНЕМ!
АКТИВНОСТЬ БУДЕТ СЧИТАТЬСЯ, ОПИРАЯСЬ НА ЧАТ**

Обо мне

**Кирилл
Сысоев**

6+ лет в Big Data

Lead Data Engineer
Sber (LLM Team)

HSE University

Hadoop, Spark, ClickHouse, Kafka, Kubernetes
Python/Scala, SQL



t.me/KRSysoev
ksysoev@hse.ru

ТG-канал
«Кузница данных»

Темы лекций

1-2) Синтаксис Python

3-4) ООП и концепции тестирования

5-6) Bash и Git

7) Модули, файловая система и исключения в Python

8-9) Docker

10-11) BigData и Hadoop

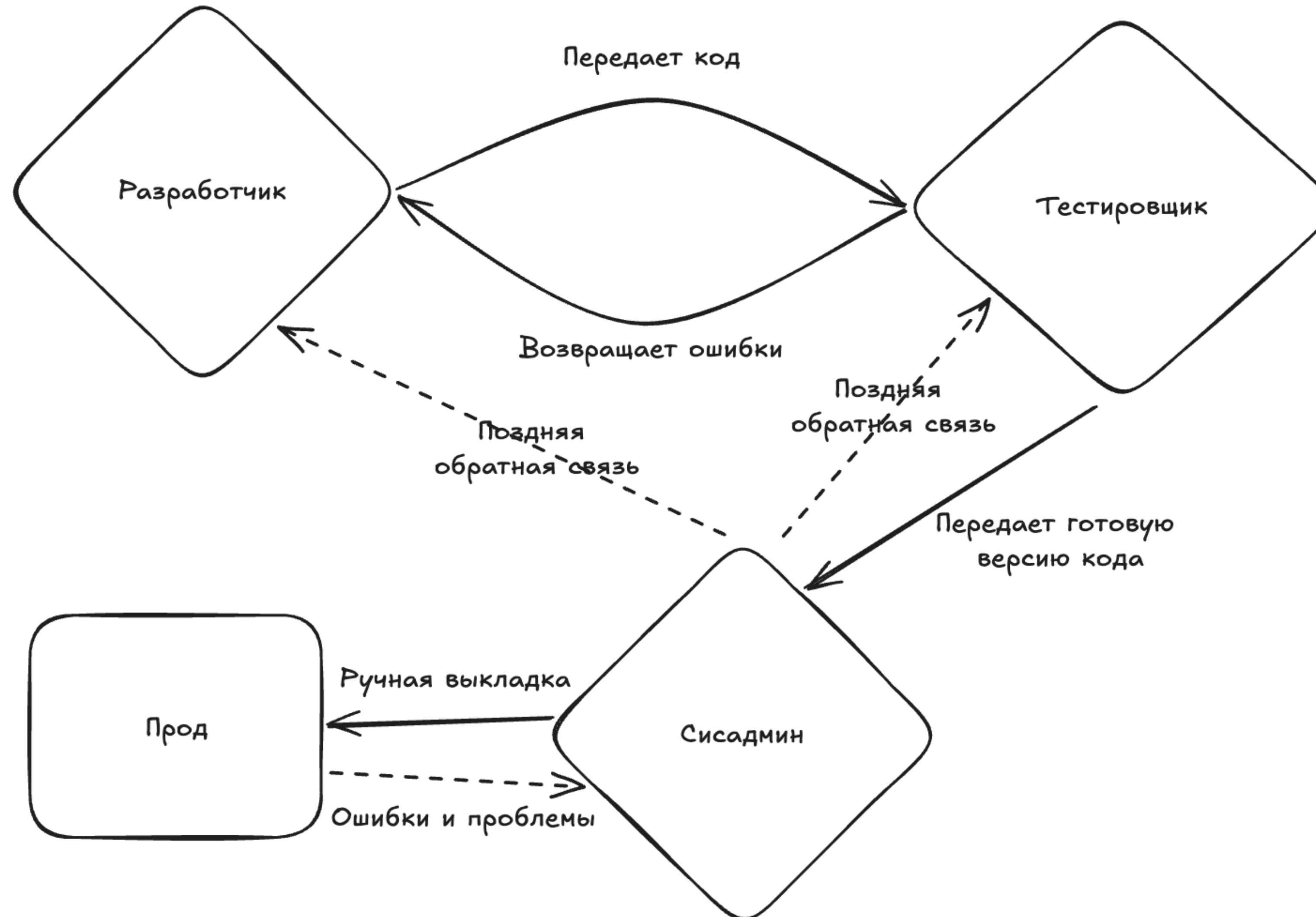
12) DevOps

Цель лекции

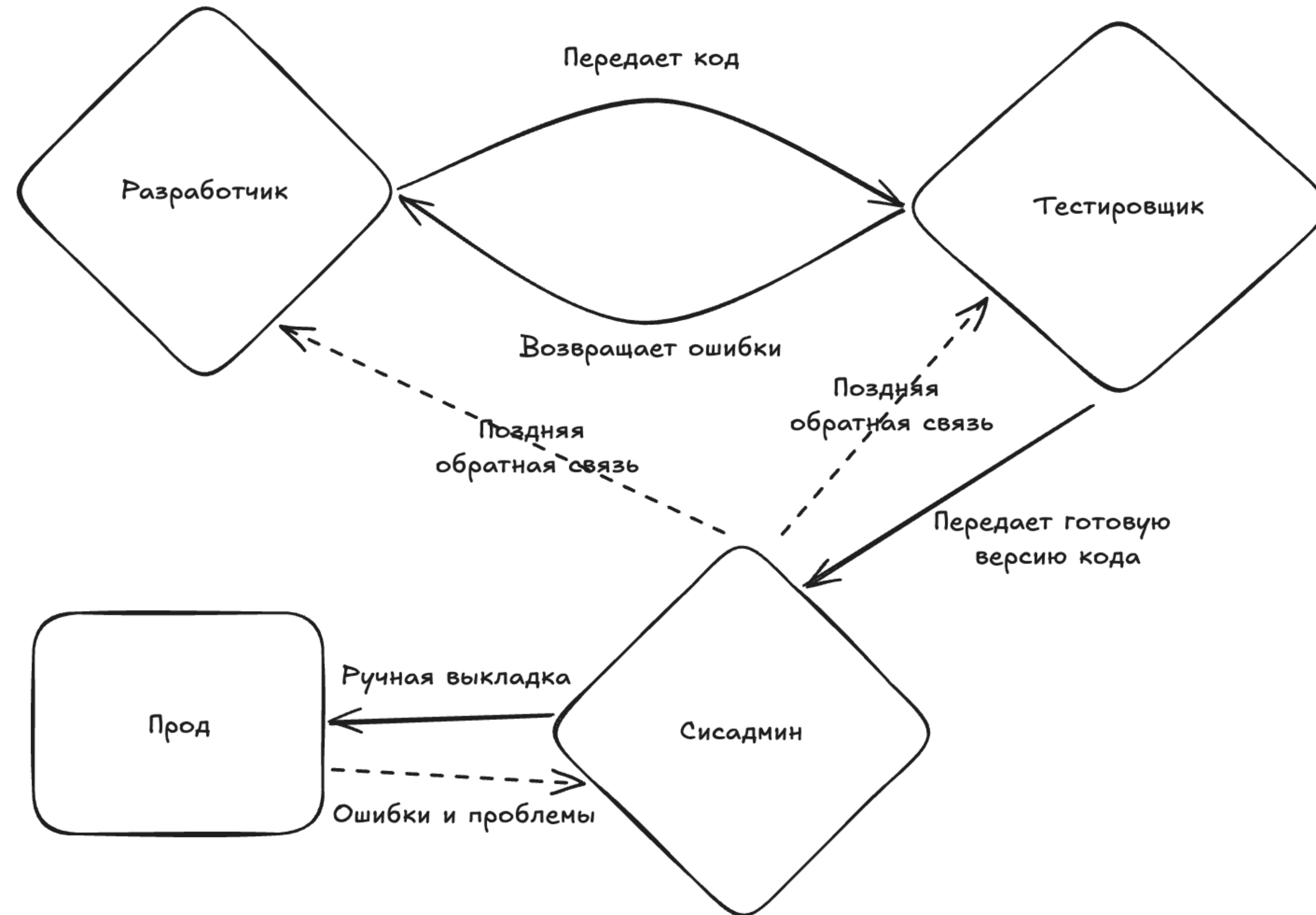
1. Что такое DevOps и зачем он нужен
2. CI/CD как процессы в разработке
3. Важность инфраструктуры
4. Наблюдаемость

Кто такой DevOps инженер

Как работало раньше



Как работало раньше



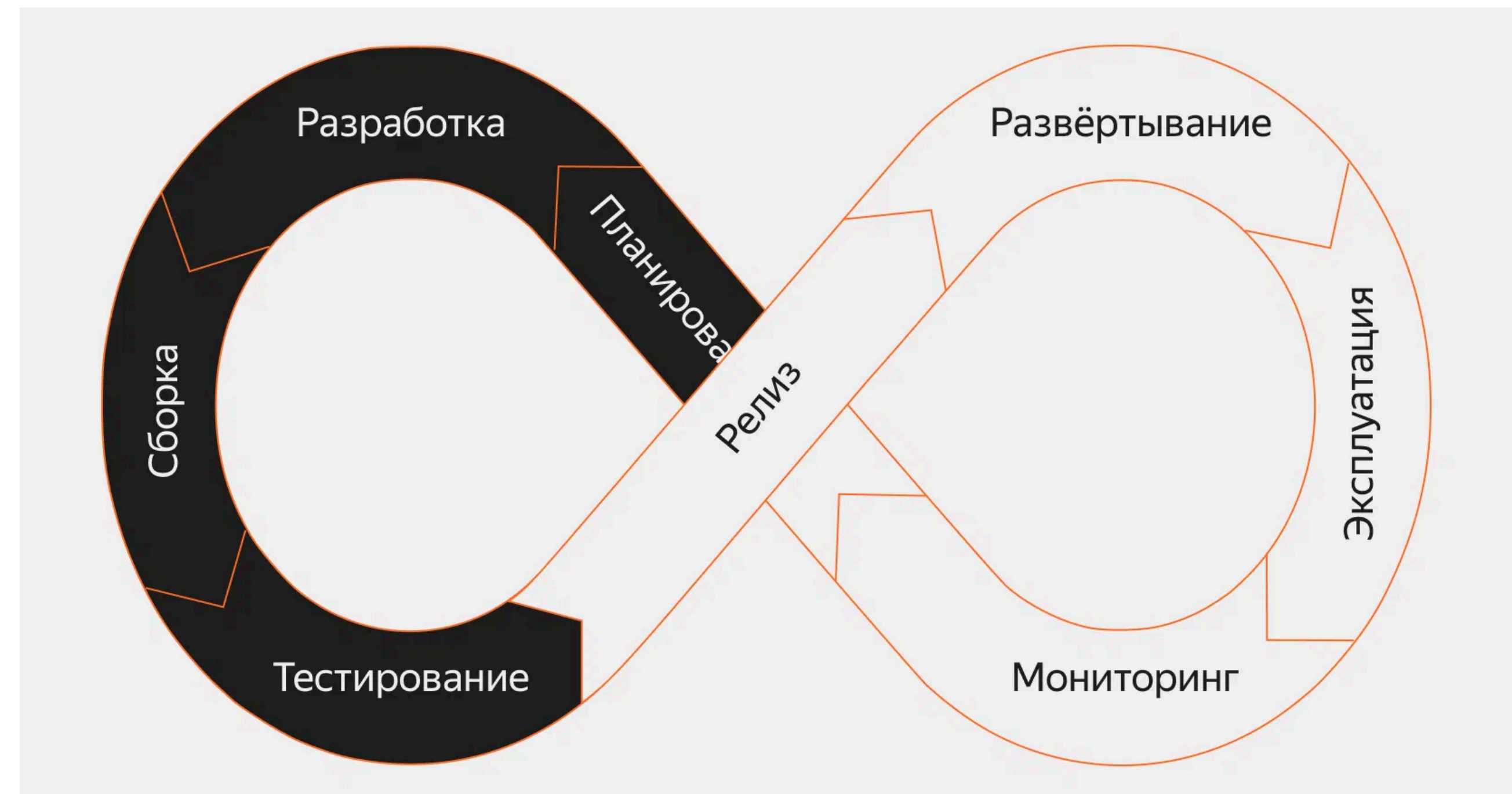
Возникла необходимость: Выпускать обновления быстро, часто и непрерывно

Решение: DevOps

Чтобы превратить разработку в единый цикл, создали целую культуру – **DevOps**.

Термин появился из слияния двух слов: **development** и **operations**, то есть «разработка» и «администрирование».

DevOps подразумевает прямое общение разработчиков, тестировщиков и сисадминов, общую ответственность, единые показатели успешности разработки и максимальную автоматизацию с помощью новых инструментов и особых подходов к работе.



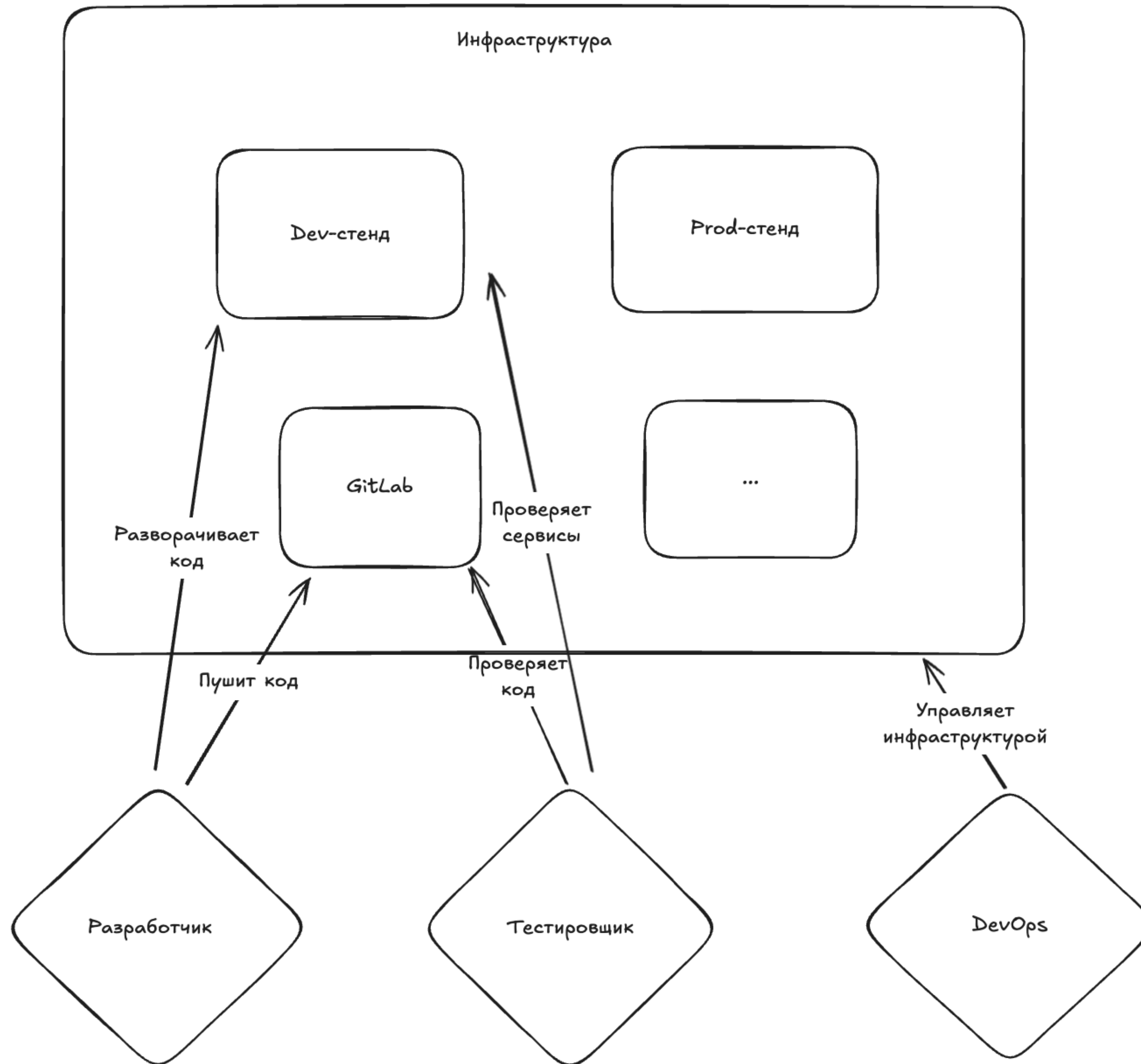
DevOps-инженер

DevOps-инженер – это ИТ-специалист, который объединяет этапы разработки программного обеспечения и его эксплуатации.

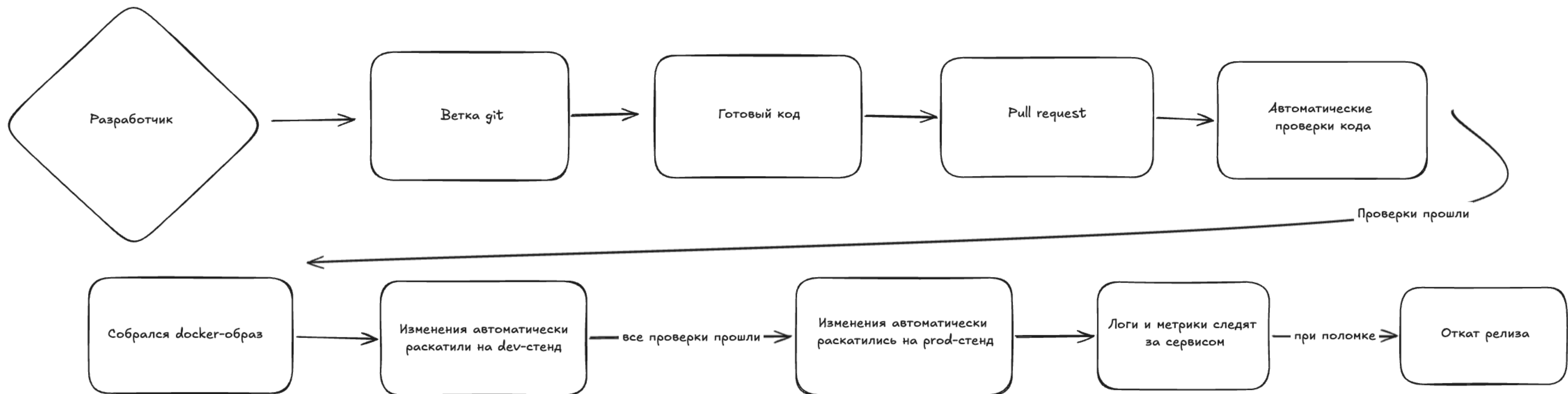
В небольших компаниях DevOps-инженер работает скорее как системный администратор. В более крупных может работать отдельно, налаживая процессы в целом и помогая коммуникациям и техническому взаимодействию разных отделов. Задача DevOps-инженера — сделать так, чтобы код непрерывно и без задержек проходил весь жизненный цикл.

Чем занимается:

- автоматизирует сборку, тестирование и доставку кода
- настраивает окружения: dev, test, stage, prod
- управляет инфраструктурой
- следит за мониторингом, логами, алертами
- помогает разработчикам быстрее и безопаснее выпускать изменения
- снижает ручной труд



Жизненный цикл изменения в коде



Зачем так усложнять выкатку релиза?

В промышленной разработке **нельзя** просто “залить код на сервер”.
Нужно убедиться, что:

- код проверен автоматически
- тесты прошли
- сборка воспроизводима
- изменение сначала проверено на тестовом стенде
- релиз одобрен командой
- production работает стабильно
- если что-то сломалось, можно быстро откатиться

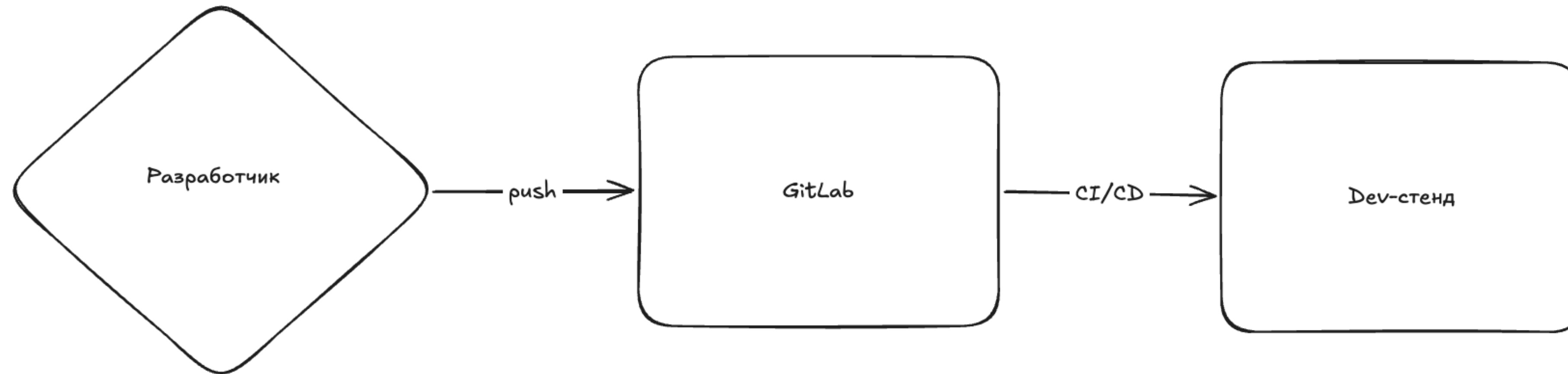
Главная идея: каждое изменение проходит понятный контролируемый путь от ноутбука разработчика до production.

Процесс CI/CD

CI/CD

CI/CD – это набор практик в разработке ПО, объединяющий непрерывную интеграцию (Continuous Integration) и непрерывную поставку/развёртывание (Continuous Delivery/Deployment).

Эта система автоматически проверяет, собирает и выкатывает обновления кода, исключая ручные ошибки и ускоряя выпуск программных продуктов.



1. Разработчик пушит код в GitLab
2. GitLab запускает pipeline
3. CI проверяет код: линтеры, тесты, сборка
4. Если все успешно, собирается артефакт или Docker-образ
5. CD автоматически выкатывает новую версию на dev-стенд
6. Команда может сразу проверить изменения в окружении, похожем на реальную систему

CI

CI (Continuous Integration) – это автоматическая проверка каждого изменения в коде перед тем, как оно попадет в основную ветку.

Что обычно делает CI:

- Запускает тесты
Проверяет, что новая логика работает и старый функционал не сломался.
- Проверяет стиль кода линтерами
Например, находит неиспользуемые переменные, плохие импорты, потенциальные ошибки.
- Запускает форматтеры или проверяет форматирование
Следит, чтобы код был оформлен единообразно у всей команды.
- Проверяет типы
Предназначенные инструменты находят ошибки в типах до запуска приложения.
- Делает security checks
Ищет уязвимые зависимости, секреты в коде, небезопасные конструкции.
- Собирает проект
Проверяет, что приложение вообще можно собрать в рабочий артефакт или Docker-образ.
- Проверяет совместимость с основной веткой
Убеждается, что изменение не ломает main / master и его можно безопасно мержить.

Главная идея CI: каждый push автоматически проходит технический контроль качества.

CD

CD (Continuous Delivery / Continuous Deployment) – это автоматическая доставка проверенного кода в окружения: dev, test, stage или production.

Что обычно делает CD:

- Берет результат CI
Использует уже проверенный код, артефакт или Docker-образ.
- Публикует артефакт
Например, кладет Docker-образ в registry.
- Выкатывает новую версию на стенд
Обновляет приложение в dev/test/stage/prod окружении.
- Применяет конфигурации
Подставляет переменные окружения, настройки, секреты, параметры подключения.
- Запускает миграции
При необходимости обновляет схему базы данных.
- Проверяет успешность выкладки
Например, health check: приложение запустилось и отвечает.
- Поддерживает rollback
Если новая версия сломалась, позволяет быстро откатиться на предыдущую.

Главная идея CD: если CI доказал, что код рабочий, CD доставляет его туда, где его можно использовать.

Популярные инструменты CI/CD

- GitLab CI/CD — встроен в GitLab, часто используют в компаниях
- GitHub Actions — CI/CD прямо внутри GitHub
- Jenkins — старый, гибкий и очень распространенный инструмент
- TeamCity — популярный корпоративный CI/CD от JetBrains
- CircleCI — облачный CI/CD-сервис
- Azure DevOps Pipelines — часто используется в Microsoft/Azure-экосистеме
- Bitbucket Pipelines — CI/CD внутри Bitbucket
- Argo CD — популярный инструмент для выкладки в Kubernetes через GitOps
- Flux CD — тоже GitOps-инструмент для Kubernetes
- Tekton — CI/CD-фреймворк для Kubernetes

All 1,000+ Finished Branches Tags

CI lint

Run pipeline

Filter pipelines



Show Pipeline ID ▾

Status	Pipeline	Triggerer	Stages	
passed 00:01:09 51 minutes ago	Merge branch 'mk-change-ruby2-channel' into 'ru... #873915908 ruby2-sync -> d285488f Scheduled latest			▾
running In progress	Scheduled Ruby 3.0 master branch pipeline #873900101 master -> 0ba5e520 Scheduled latest		→	▾ ▾
passed 00:01:04 2 hours ago	Merge branch 'mk-change-ruby2-channel' into 'ru... #873879309 ruby2-sync -> d285488f Scheduled latest			▾
failed 02:50:19 1 hour ago	Scheduled Ruby 3.0 master branch pipeline #873854906 master -> 0ba5e520 Scheduled latest		→	▾ ▾
passed 01:17:22 3 hours ago	Ruby 3.0 merged_result MR pipeline #873841875 120160 -> f930a736 latest		→	▾ ▾

Тест Docker

7 минут

[https://forms.yandex.ru/u/
6a16ff6949363941e8432a3d](https://forms.yandex.ru/u/6a16ff6949363941e8432a3d)



Docker Compose

Docker Compose — это инструмент для определения и запуска многоконтейнерных Docker-приложений.

- Один файл конфигурации (`docker-compose.yml`), в котором описываются все сервисы, их образы, порты, переменные окружения, диски (`volumes`) и сети.
- Одна команда (`docker compose up`) запускает всё приложение целиком.
- Управление зависимостями — можно указать, какой сервис от какого зависит, тогда Compose запустит их в правильном порядке.

Что можно почитать? (Docker)

- <https://habr.com/ru/companies/netologyru/articles/967546/>
- <https://habr.com/ru/articles/253877/>
- <https://selectel.ru/blog/what-is-docker/>
- <https://habr.com/ru/articles/963180/>
- <https://tproger.ru/articles/dockerize-python-sozdajom-obraz-docker-iz-prilozhenija-na-python>

Спасибо за внимание!



t.me/TheDataForge



СЫСОВЕВ КИРИЛЛ

t.me/KRSysoev

